UNIVERSIDAD AUTÓNOMA DE CHIRIQUÍ

Facultad de Economía · Escuela de Ciencias de Computación

LABORATORIO #4

Plataforma de Pedidos de Comida a Domicilio

Selección y justificación del patrón arquitectónico · Programación V — PROG 309

Arian Cervantes · Jairo Chávez · Débora Cisneros · Daniela Pittí

Prof. Alexis Flores · II Semestre · Fecha de entrega: 11/09/2026

1. OBJETIVO

Objetivo del laboratorio

Analizar **el caso de una plataforma de pedidos de comida a domicilio,** seleccionar el patrón arquitectónico (o combinación) más adecuado entre los diez patrones estudiados, justificar la decisión frente a las alternativas descartadas, y elaborar un diagrama de alto nivel para su defensa oral.

S **Seleccionar**

el patrón óptimo entre 10 patrones arquitectónicos

J **Justificar**

la decisión por escrito frente a alternativas descartadas

D **Diagramar**

la arquitectura propuesta a alto nivel

2. EL CASO

Resumen del caso de estudio

Una empresa emergente lanza una app web y móvil para pedidos de comida a domicilio; restaurantes reciben pedidos en tiempo real y repartidores son notificados automáticamente.

1

Reutilización de lógica

Cálculo de precios, comisiones y promociones reutilizado entre app web, app móvil y un futuro panel administrativo.

2

Escalabilidad

El sistema debe atender picos de pedidos en horas de almuerzo y cena sin degradar el servicio.

3

Notificación desacoplada

Cada cambio de estado del pedido debe notificar a cliente, restaurante y repartidor sin que se conozcan entre sí.

4

Independencia visual

El equipo debe poder cambiar el diseño visual sin modificar la lógica de negocio ni la base de datos.

3.1 CLASIFICACIÓN

De requerimiento a patrón candidato

Requerimiento	Aspecto arquitectónico	Patrón(es) candidato(s)
Reutilización de lógica de precios/comisiones/promociones entre 3 clientes	Separación de responsabilidades y reutilización de lógica	**Capas**
Escalar ante picos de pedidos (almuerzo/cena) sin degradar el servicio	Acceso de múltiples clientes a un servicio centralizado escalable	**Cliente-Servidor**
Notificación de cambios de estado a módulos que no se conocen entre sí	Comunicación desacoplada entre componentes	**Bus de eventos (y, alt., Broker)**
Cambiar el diseño visual sin tocar lógica de negocio ni base de datos	Independencia entre interfaz y lógica	**MVC**

3.2 SELECCIÓN

Una combinación de cuatro patrones

Cada patrón resuelve una parte específica del problema, en lugar de forzar uno solo para todo el sistema.

Cliente-Servidor

Topología general

App web, móvil y panel admin consumen un servicio centralizado vía API; permite escalar el servidor de forma independiente en picos.

Capas

Dentro del servidor central

Separa presentación (API REST), lógica de negocio (precios/comisiones) y datos, permitiendo reutilizar la lógica entre los 3 clientes.

Bus de eventos

Notificación de estados

Cada cambio de estado se publica como evento; cliente, restaurante y repartidor se suscriben sin conocerse entre sí.

MVC

Dentro de cada app cliente

Separa la Vista del Modelo/Controlador, permitiendo rediseñar la interfaz sin tocar lógica de negocio ni base de datos.

3.3 COMPARACIÓN

Por qué esta combinación es superior

Frente a usar un único patrón, la combinación cubre los cuatro requerimientos con el menor acoplamiento posible.

Capas	Reutilización de lógica de negocio entre los tres clientes
Cliente-Servidor	Acceso centralizado y escalable ante picos de demanda
MVC	Independencia de la interfaz visual
Bus de eventos	Notificación entre módulos que no deben conocerse

Ningún patrón individual del listado de diez cubre, por sí solo, los cuatro frentes a la vez.

3.4 DESCARTADOS

Patrones considerados y descartados

Broker

Desacople similar al Bus de eventos, pero exige infraestructura de descubrimiento más compleja de la necesaria.

Maestro-esclavo

No hay tareas idénticas para paralelizar; procesar un pedido es un flujo secuencial de estados.

Pipe-filter

No hay datos que fluyan por etapas de transformación; varía el estado de una misma entidad (el pedido).

Peer-to-peer

El negocio necesita control centralizado sobre pagos, inventario y consistencia de datos.

Blackboard

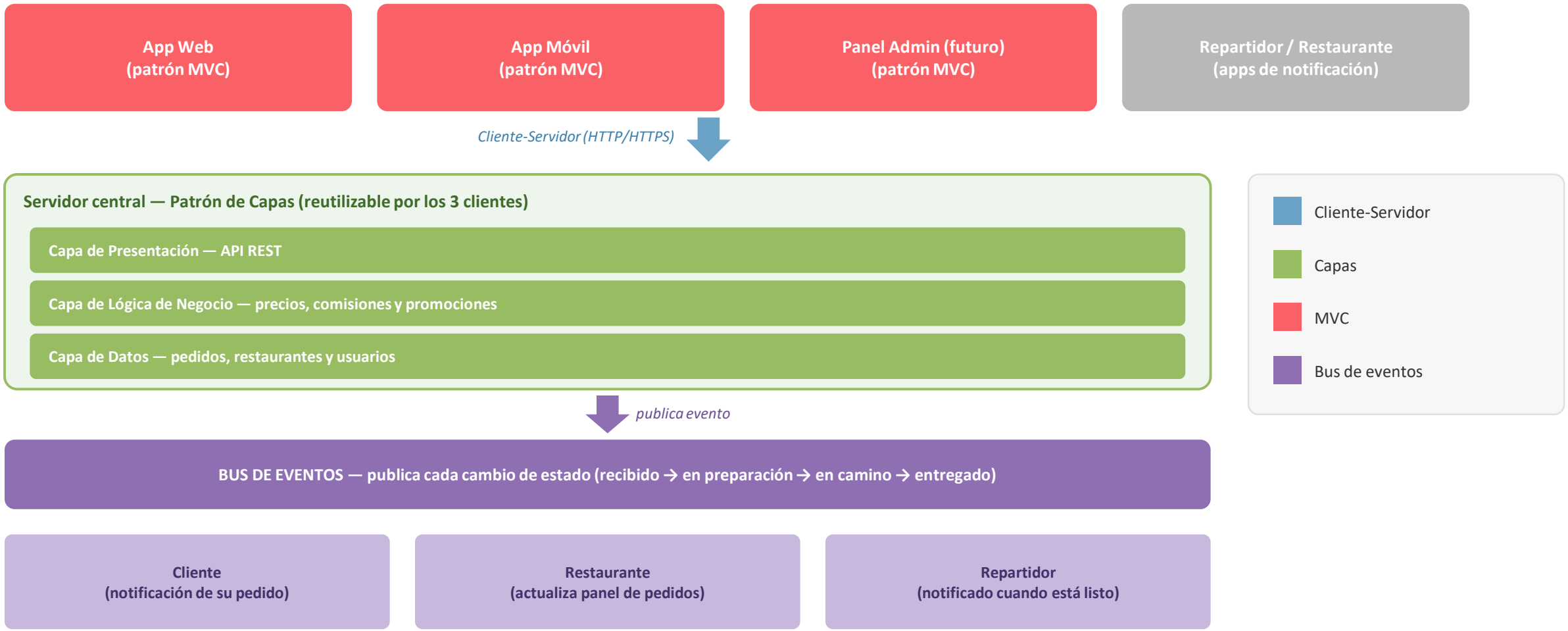
El proceso sigue reglas de negocio bien definidas, no una búsqueda exploratoria de solución.

Intérprete

No existe un lenguaje o gramática propia que el sistema deba evaluar en el flujo de pedidos.

4. DIAGRAMA

Arquitectura de alto nivel



Preguntas de análisis (I)

5.1 ¿Qué patrón(es) eligió su equipo y por qué es superior a las alternativas?

El equipo eligió la combinación Cliente-Servidor + Capas + MVC + Bus de eventos. Es superior porque cada requerimiento del caso encuentra respuesta en el patrón diseñado específicamente para ese tipo de problema, en vez de forzar un solo patrón a cubrir necesidades para las que no fue pensado.

5.2 ¿Qué requerimiento fue el más determinante para la decisión arquitectónica?

La notificación de cada cambio de estado a varios módulos (cliente, restaurante, repartidor) sin que se conozcan entre sí. Es el único requerimiento que exige explícitamente comunicación desacoplada entre múltiples receptores independientes, apuntando de forma directa al Bus de eventos.

Preguntas de análisis (II)

5.3

Si además hubiera un asistente de voz, ¿qué patrón adicional consideraría?

Intérprete

Un asistente de voz debe convertir una instrucción hablada ("pide dos hamburguesas del restaurante X") en una estructura de datos procesable. Esto exige una gramática simple y estable (verbos de acción, productos, cantidades) evaluada por un componente dedicado — exactamente el problema que resuelve el patrón Intérprete.

5.4

¿Qué patrón de los diez sería claramente inadecuado para este caso?

Peer-to-peer

El caso exige control centralizado sobre pagos, inventario de restaurantes y consistencia del estado de los pedidos. Un modelo donde todos los nodos cooperan como iguales, sin un punto único de control, dificultaría garantizar la seguridad transaccional y la coherencia de los datos.

5.5 Escenario de bajo volumen y una sola ciudad

¿Cómo cambiaría la decisión si la empresa solo esperara operar en una sola ciudad con bajo volumen de pedidos?

Se mantiene	Se simplifica	Se mantiene
Capas	**Bus de eventos → llamadas directas**	**MVC**
Se conserva la separación de responsabilidades y la reutilización de lógica de negocio.	El desacople se aprovecha más en escenarios de mayor escala y variedad de suscriptores; con bajo volumen bastan llamadas síncronas.	La necesidad de cambiar el diseño visual sin tocar la lógica de negocio no depende del volumen de pedidos.

Conclusiones

1. Ningún patrón arquitectónico aislado resuelve por sí solo todos los requerimientos de un sistema real; lo relevante es qué combinación responde mejor al caso.

2. Cliente-Servidor + Capas + MVC + Bus de eventos atiende simultáneamente reutilización de lógica, escalabilidad, independencia visual y notificación desacoplada.

3. Descartar explícitamente Peer-to-peer, Blackboard, Intérprete, Pipe-filter y Maestro-esclavo evidencia que la decisión fue deliberada y no la única alternativa considerada.

4. Escenarios futuros (asistente de voz) o cambios de escala (una sola ciudad) confirman que la arquitectura debe revisarse conforme evolucionan los requerimientos.

Referencias

Huaman, W. C. (2018). Los 10 patrones comunes de arquitectura de software. Medium.

Blancarte, O. (2020). Introducción a la arquitectura de software.

¡Gracias!